

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4
Marks:40

Total

Annexure A

CASE STUDY

Code of Conduct:

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

- I. If a patient has Fever AND Cough $\rightarrow$ Possible Flu
- II. If a patient has Fever AND Rash $\rightarrow$ Possible Measles
- III. If a patient has Cough AND Breathlessness $\rightarrow$ Possible Pneumonia
- IV. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

- $\forall x: \text{Vehicle}(x) \wedge \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$ [Rule 1]
- $\forall x: \text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$ [Rule 2]
- $\forall x \forall y: \text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$ [Rule 3]
- Facts: $\text{Vehicle}(\text{Ambulance}), \text{EmergencyType}(\text{Ambulance}), \text{Vehicle}(\text{CarA}), \text{Behind}(\text{CarA}, \text{Ambulance})$

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1. P1: $\text{SoilDry} \rightarrow \text{IrrigationNeeded}$
2. P2: $\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$
3. P3: $\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$
4. Facts: $\text{SoilDry} = \text{True}, \text{CropWheat} = \text{True}$

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF).

Show each conversion step (eliminate implication $\rightarrow$ move $\neg$ inward $\rightarrow$ distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause $\square$. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

- Stench [1,2] $\rightarrow$ Wumpus is adjacent to cell [1,2]
- Breeze [1,1] $\rightarrow$ Pit is adjacent to cell [1,1]
- Glitter [x,y] $\rightarrow$ Gold is at [x,y]
- $\neg$ Wumpus [x,y] $\wedge$ $\neg$ Pit[x,y] $\rightarrow$ Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] $\rightarrow$ [1,2] $\rightarrow$ [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding ; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The knowledge base contains rules connecting symptoms to possible diseases and facts about Patient A.

(a) Representation Using Propositional Logic

Step 1: Define the Propositional Symbols

Let the following propositions represent the facts:

- F = Patient A has Fever.
- C = Patient A has Cough.
- R = Patient A has Rash.
- B = Patient A has Breathlessness.
- Flu = Patient A has Possible Flu.
- M = Patient A has Possible Measles.
- P = Patient A has Possible Pneumonia.

Step 2: Represent the Rules

Rule I: Fever AND Cough $\rightarrow$ Possible Flu

[
 $F \wedge C \rightarrow Flu$
]

Rule II: Fever AND Rash $\rightarrow$ Possible Measles

[
 $F \wedge R \rightarrow M$
]

Rule III: Cough AND Breathlessness $\rightarrow$ Possible Pneumonia

[
 $C \wedge B \rightarrow P$
]

Step 3: Represent Patient A's Facts

Given:

Fever = True

Cough = True

Breathlessness = True

Therefore:

```

[
F = True
]
[
C = True
]
[
B = True
]
There is no fact stating that Patient A has a rash, so:
[
R = False/Unknown
]
The complete knowledge base can therefore be represented as:
[
KB = {F \land C \rightarrow Flu,; F \land R \rightarrow M,; C \land B \rightarrow P,; F,; C,; B}
]

```

(b) Forward Chaining

Forward Chaining starts with the known facts and repeatedly applies rules whose conditions are satisfied.

Initial Facts

```

[
F,; C,; B
]
That means:
Fever = True
Cough = True
Breathlessness = True

```

Inference Step 1

Check Rule I:

```

[
F \land C \rightarrow Flu
]

```

We know:

```

[
F = True
]

```

and

```

[
C = True
]

```

Therefore, both conditions are satisfied.

Rule I fires:

[
F $\wedge$ C $\rightarrow$ Flu
]
Derived fact:
[
Flu = True
]
So Patient A has Possible Flu.

Inference Step 2

Check Rule II:
[
F $\wedge$ R $\rightarrow$ M
]
We know:
[
F = True
]
But there is no fact that:
[
R = True
]
Therefore, Rule II cannot fire.
Possible Measles cannot be derived.

Inference Step 3

Check Rule III:
[
C $\wedge$ B $\rightarrow$ P
]
We know:
[
C = True
]
and
[
B = True
]
Therefore, both conditions are satisfied.
Rule III fires:
[
C $\wedge$ B $\rightarrow$ P
]
Derived fact:
[
P = True
]
So Patient A has Possible Pneumonia.

Forward Chaining Table

Step	Rule Checked	Required Facts	Result
0	—	F, C, B	Initial facts
1	Rule I	$F \wedge C$	Flu derived
2	Rule II	$F \wedge R$	Not fired — R unavailable
3	Rule III	$C \wedge B$	Pneumonia derived

Final Conclusions

The system derives:

[
\boxed{Possible\ Flu}
]

and

[
\boxed{Possible\ Pneumonia}
]

Possible Measles is not derived because the Rash fact is unavailable.

(c) Backward Chaining for Pneumonia

The goal is:

[
\boxed{P}
]

where P = Patient A has Possible Pneumonia.

Backward Chaining starts with the goal and works backward to determine what facts are required to prove it.

Step 1: Start with the Goal

[
Goal: P
]

Search the knowledge base for a rule that concludes P.

The matching rule is:

[
 $C \wedge B \rightarrow P$
]

Therefore, to prove P, we need to prove:

[
 $C \wedge B$
]

So the goal is reduced to two sub-goals:

[
C
]
and


```
[
B
]
```

Step 2: Verify Cough

We have the fact:

```
[
C = True
]
```

Therefore:

```
[
\boxed{C;proved}
]
```

Step 3: Verify Breathlessness

We have the fact:

```
[
B = True
]
```

Therefore:

```
[
\boxed{B;proved}
]
```

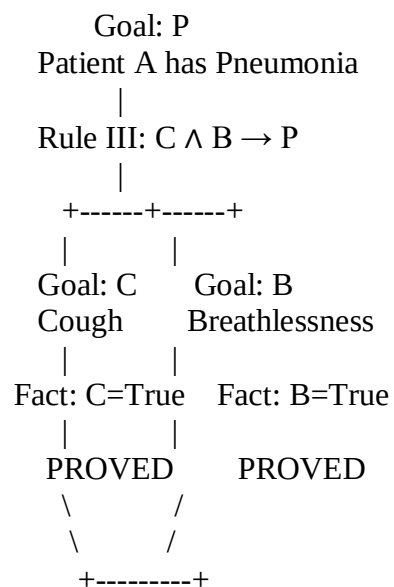
Since both required conditions are true:

```
[
C \land B = True
]
```

Therefore:

```
[
\boxed{P = True}
]
```

Goal Reduction Tree



$$\begin{array}{c}
 | \\
 P = \text{TRUE} \\
 | \\
 \text{Pneumonia Goal PROVED}
 \end{array}$$

Final Answer

Backward Chaining successfully proves:

```
[
\boxed{C \wedge B \rightarrow P}
]
```

Since both Cough and Breathlessness are known facts for Patient A:

```
[
\boxed{Patient\ A\ has\ Possible\ Pneumonia}
]
```

Therefore, the goal is successfully verified.

Important: In a real medical system, these rules should be treated as preliminary decision support only. The logical conclusion “Possible Pneumonia” is not equivalent to a confirmed medical diagnosis.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a First-Order Logic (FOL) knowledge base to manage emergency vehicle routing.

The given knowledge base is:

Rule 1:

```
[
\forall x;(Vehicle(x)\wedge EmergencyType(x)\rightarrow ClearPath(x))
]
```

Rule 2:

```
[
\forall x;(ClearPath(x)\rightarrow GreenSignal(x)\wedge \neg RedSignal(x))
]
```

Rule 3:

```
[
\forall x\forall y;(Vehicle(x)\wedge Behind(x,y)\wedge GreenSignal(y)\rightarrow Proceed(x))
]
```

Facts:

```
[
Vehicle(Ambulance)
]
[
EmergencyType(Ambulance)
]
[
Vehicle(CarA)
]
[
Behind(CarA,Ambulance)
]
```

(a) Unification of Rule 1 with the Given Facts

Rule 1

```
[  
\forall x;(Vehicle(x)\land EmergencyType(x)\rightarrow ClearPath(x))  
]
```

The antecedent contains:

```
[  
Vehicle(x)\land EmergencyType(x)  
]
```

We need to match this with the facts:

```
[  
Vehicle(Ambulance)  
]
```

and

```
[  
EmergencyType(Ambulance)  
]
```

Step 1: Unify Vehicle(x)

Compare:

```
[  
Vehicle(x)  
]
```

with:

```
[  
Vehicle(Ambulance)  
]
```

The predicate names are the same, and therefore:

```
[  
x = Ambulance  
]
```

So the substitution is:

```
[  
\theta_1 = {x/Ambulance}  
]
```

Step 2: Unify EmergencyType(x)

Substitute (x=Ambulance):

```
[  
EmergencyType(Ambulance)  
]
```

Compare with the fact:

```
[  
EmergencyType(Ambulance)  
]
```

They match exactly.

Therefore:

```
[
\theta_2 = \{x/Ambulance\}
]
```

Final MGU

The Most General Unifier (MGU) is:

```
[
\boxed{\theta = \{x/Ambulance\}}
]
```

After applying the substitution to Rule 1:

```
[
Vehicle(Ambulance) \wedge EmergencyType(Ambulance)
\rightarrow ClearPath(Ambulance)
]
```

Since both premises are true:

```
[
\boxed{ClearPath(Ambulance)}
]
```

is derived.

(b) Proving Proceed(CarA) Using Resolution

The goal is:

```
[
\boxed{Proceed(CarA)}
]
```

Resolution proves a statement by adding its negation to the knowledge base and deriving a contradiction.

Therefore, negate the goal:

```
[
\neg Proceed(CarA)
]
```

If this leads to a contradiction, the original goal is proved.

Step 1: Convert Rules to CNF

Rule 1

Original:

```
[
Vehicle(x) \wedge EmergencyType(x) \rightarrow ClearPath(x)
]
```

Using:

```
[
A \rightarrow B \equiv \neg A \vee B
]
```

we get:

[
 $\boxed{\neg \text{Vehicle}(x) \vee \neg \text{EmergencyType}(x) \vee \text{ClearPath}(x)}$
]
 Call this C1.

Rule 2

Original:

[
 $\text{ClearPath}(x) \rightarrow \text{GreenSignal}(x) \wedge \neg \text{RedSignal}(x)$
]

Because the consequent contains an AND, it becomes two implications:

[
 $\text{ClearPath}(x) \rightarrow \text{GreenSignal}(x)$
]

and

[
 $\text{ClearPath}(x) \rightarrow \neg \text{RedSignal}(x)$
]

Therefore, the CNF clauses are:

[
 $\boxed{\neg \text{ClearPath}(x) \vee \text{GreenSignal}(x)}$
]

C2

and

[
 $\boxed{\neg \text{ClearPath}(x) \vee \neg \text{RedSignal}(x)}$
]

C3

For proving $\text{Proceed}(\text{CarA})$, only C2 is required.

Rule 3

Original:

[
 $\text{Vehicle}(x) \wedge \text{Behind}(x,y) \wedge \text{GreenSignal}(y) \rightarrow \text{Proceed}(x)$
]

CNF:

[
 $\boxed{\neg \text{Vehicle}(x) \vee \neg \text{Behind}(x,y) \vee \neg \text{GreenSignal}(y) \vee \text{Proceed}(x)}$
]

Call this C4.

Step 2: Relevant Facts

The relevant facts are:

C5

```
[  
Vehicle(Ambulance)  
]
```

C6

```
[  
EmergencyType(Ambulance)  
]
```

C7

```
[  
Vehicle(CarA)  
]
```

C8

```
[  
Behind(CarA,Ambulance)  
]
```

And the negated goal:

C9

```
[  
\neg Proceed(CarA)  
]
```

Step 3: Derive ClearPath(Ambulance)

Use C1:

```
[  
\neg Vehicle(x)\lor\neg EmergencyType(x)\lor ClearPath(x)  
]
```

Resolve with:

```
[  
Vehicle(Ambulance)  
]
```

Using:

```
[  
\theta_1=\{x/Ambulance\}  
]
```

we obtain:

```
[  
\neg EmergencyType(Ambulance)\lor ClearPath(Ambulance)  
]
```

Resolve with:

```
[  
EmergencyType(Ambulance)  
]
```

Therefore:

```
[  
\boxed{ClearPath(Ambulance)}  
]
```

Step 4: Derive GreenSignal(Ambulance)

Use C2:

```
[  
\neg ClearPath(x)\lor GreenSignal(x)  
]
```

Resolve with:

```
[  
ClearPath(Ambulance)  
]
```

Using:

```
[  
\theta_2=\{x/Ambulance\}  
]
```

we obtain:

```
[  
\boxed{GreenSignal(Ambulance)}  
]
```

Step 5: Use Rule 3

C4 is:

```
[  
\neg Vehicle(x)\lor  
\neg Behind(x,y)\lor  
\neg GreenSignal(y)\lor  
Proceed(x)  
]
```

We have:

```
[  
Vehicle(CarA)  
]  
[  
Behind(CarA,Ambulance)  
]  
[  
GreenSignal(Ambulance)  
]
```

Use the substitution:

```
[  
\theta_3=\{x/CarA,y/Ambulance\}  
]
```

After resolving with the three facts, we obtain:

```
[  
\boxed{Proceed(CarA)}  
]
```

Step 6: Resolve with the Negated Goal

We initially added:

```
[  
\neg Proceed(CarA)  
]
```

Now we have:

```
[  
Proceed(CarA)  
]
```

Resolving:

```
[  
Proceed(CarA)  
]
```

with

```
[  
\neg Proceed(CarA)  
]
```

produces:

```
[  
\boxed{\square}  
]
```

where ($\square$) represents the empty clause, meaning a contradiction has been reached.

Therefore:

```
[  
\boxed{Proceed(CarA)\text{ is proved}}  
]
```

Resolution Summary

Vehicle(Ambulance)

+

EmergencyType(Ambulance)

↓ Rule 1

ClearPath(Ambulance)

↓ Rule 2

GreenSignal(Ambulance)

Vehicle(CarA)

Behind(CarA, Ambulance)

GreenSignal(Ambulance)

↓ Rule 3

Proceed(CarA)

+

$\neg$ Proceed(CarA)

↓ Resolution

□

Hence, the goal is successfully proved.

(c) Handling Two Simultaneous Emergency Vehicles

Is the Current Knowledge Base Sufficient?

Partially, but not completely.

The existing FOL rules are written using variables, so they can technically apply to any number of vehicles.

For example, suppose there are two emergency vehicles:

```
[  
Vehicle(Ambulance1)  
]  
[  
EmergencyType(Ambulance1)  
]  
[  
Vehicle(Ambulance2)  
]  
[  
EmergencyType(Ambulance2)  
]
```

Rule 1 can independently derive:

```
[  
ClearPath(Ambulance1)  
]
```

and:

```
[  
ClearPath(Ambulance2)  
]
```

Rule 2 can then derive:

```
[  
GreenSignal(Ambulance1)  
]
```

and:

```
[  
GreenSignal(Ambulance2)  
]
```

Therefore, the FOL representation is general enough to recognize multiple emergency vehicles.

However, it does not contain rules for resolving conflicts between two emergency vehicles.

Missing Information

Consider:

Ambulance1 → Intersection X

Ambulance2 → Intersection X

Both may require a green signal at the same intersection.

The current knowledge base does not specify:

Which emergency vehicle gets priority.

What happens if both require the same signal.

Whether one vehicle has higher urgency.

How conflicting routes are resolved.

How to prevent collisions.

How to coordinate signals for both vehicles.

Whether a path should remain clear for one vehicle before allowing another.

Therefore, additional rules and facts are needed.

Additional Rule 1: Emergency Priority

A priority value can be introduced.

For example:

```
[  
\forall x \forall y;  
Emergency(x) \land PriorityHigher(x,y)  
\rightarrow GivePriority(x)  
]
```

This allows the system to determine which emergency vehicle should receive priority.

Additional Rule 2: Route Conflict

The system should represent whether two vehicles require the same route:

```
[  
\forall x \forall y;  
Emergency(x) \land Emergency(y) \land SameRoute(x,y)  
\rightarrow RouteConflict(x,y)  
]
```

This allows the controller to detect competing routes.

Additional Rule 3: Conflict Resolution

A rule can specify that a lower-priority vehicle should wait when a higher-priority vehicle requires the same path:

```
[  
\forall x \forall y;  
RouteConflict(x,y) \land PriorityHigher(x,y)  
\rightarrow Hold(y)  
]
```

Thus, if Ambulance1 has higher priority than Ambulance2:

```
[  
PriorityHigher(Ambulance1,Ambulance2)  
]
```

and both require the same route:

```
[  
RouteConflict(Ambulance1,Ambulance2)  
]
```

then:

```
[  
Hold(Ambulance2)  
]
```

can be derived.

Additional Facts

The knowledge base would also need facts such as:

```
[
Emergency(Ambulance1)
]
[
Emergency(Ambulance2)
]
[
Priority(Ambulance1,High)
]
[
Priority(Ambulance2,Medium)
]
[
SameRoute(Ambulance1,Ambulance2)
]
```

These facts allow the system to reason about conflicts.

Final Analysis

The current FOL knowledge base can independently process multiple emergency vehicles, because the rules use universally quantified variables.

However, it is not sufficient for coordinated simultaneous emergency routing because it lacks conflict-resolution and priority rules.

Therefore, the knowledge base should be extended with:

- Emergency priority information
- Route/intersection conflict facts
- Priority comparison rules
- Waiting/holding rules
- Collision-avoidance rules
- Traffic-signal coordination rules

Final Conclusion

```
[
\boxed{\text{Current KB: Handles multiple vehicles individually}}
]
but
[
\boxed{\text{Additional rules are required for simultaneous conflict-free routing}}
]
```

This extension makes the FOL agent more realistic because it can reason not only about whether an emergency vehicle should proceed, but also which vehicle should proceed first when multiple emergency vehicles compete for the same traffic resources.

Case Study 3 — Agricultural Expert Reasoning System

The agricultural advisory logical agent has the following propositional knowledge base:

P1:

```
[
```

```

SoilDry \rightarrow IrrigationNeeded
]
P2:
[
IrrigationNeeded \land CropWheat \rightarrow ApplyDripMethod
]
P3:
[
\neg ApplyDripMethod \land CropWheat \rightarrow CropAtRisk
]
Facts:
[
SoilDry = True
]
[
CropWheat = True
]

```

(a) Conversion into Conjunctive Normal Form (CNF)

CNF requires every formula to be a conjunction of clauses, where each clause contains literals connected by OR.

P1: $\text{SoilDry} \rightarrow \text{IrrigationNeeded}$

Step 1: Eliminate implication

```

Use:
[
A \rightarrow B \equiv \neg A \lor B
]
Therefore:
[
SoilDry \rightarrow IrrigationNeeded
]
becomes:
[
\neg SoilDry \lor IrrigationNeeded
]

```

CNF:

```

[
\boxed{\neg SoilDry \lor IrrigationNeeded}
]

```

P2: $\text{IrrigationNeeded} \wedge \text{CropWheat} \rightarrow \text{ApplyDripMethod}$

Step 1: Eliminate implication

Let:

[
A = $\text{IrrigationNeeded} \wedge \text{CropWheat}$
]

Then:

[
 $A \rightarrow B \equiv \neg A \vee B$
]

Therefore:

[
 $\neg(\text{IrrigationNeeded} \wedge \text{CropWheat}) \vee \text{ApplyDripMethod}$
]

Step 2: Move $\neg$ inward

Using De Morgan's law:

[
 $\neg(A \wedge B) \equiv \neg A \vee \neg B$
]

Therefore:

[
 $\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}$
]

This is already in CNF.

CNF:

[
 $\boxed{\neg \text{IrrigationNeeded} \vee \neg \text{CropWheat} \vee \text{ApplyDripMethod}}$
]

P3: $\neg \text{ApplyDripMethod} \wedge \text{CropWheat} \rightarrow \text{CropAtRisk}$

Step 1: Eliminate implication

[
 $\neg(\neg \text{ApplyDripMethod} \wedge \text{CropWheat}) \vee \text{CropAtRisk}$
]

Step 2: Move $\neg$ inward

Using De Morgan's law:

[
 $\neg(A \wedge B) = \neg A \vee \neg B$
]

we get:

```
[
\neg(\neg ApplyDripMethod)\lor\neg CropWheat\lor CropAtRisk
]
```

Since:

```
[
\neg(\neg ApplyDripMethod)=ApplyDripMethod
]
```

we obtain:

```
[
ApplyDripMethod\lor\neg CropWheat\lor CropAtRisk
]
```

CNF:

```
[
\boxed{ApplyDripMethod\lor\neg CropWheat\lor CropAtRisk}
]
```

Facts in CNF

Fact 1

```
[
SoilDry=True
]
```

is represented as the unit clause:

```
[
\boxed{SoilDry}
]
```

Fact 2

```
[
CropWheat=True
]
```

is represented as:

```
[
\boxed{CropWheat}
]
```

Complete CNF Knowledge Base

Therefore:

```
[
\boxed{
\begin{aligned}
C_1 \ \&: \ \neg SoilDry\lor IrrigationNeeded\
C_2 \ \&: \ \neg IrrigationNeeded\lor\neg CropWheat\lor ApplyDripMethod\
C_3 \ \&: \ ApplyDripMethod\lor\neg CropWheat\lor CropAtRisk\
C_4 \ \&: \ SoilDry\
C_5 \ \&: \ CropWheat
\end{aligned}
}
```

```
\end{aligned}}
]
```

(b) Resolution Refutation to Prove ApplyDripMethod

The theorem to prove is:

```
[
\boxed{ApplyDripMethod}
]
```

Resolution Refutation works by assuming the negation of the goal and showing that it produces a contradiction.

Step 1: Negate the Goal

Goal:

```
[
ApplyDripMethod
]
```

Negated goal:

```
[
\boxed{\neg ApplyDripMethod}
]
```

Add this to the CNF knowledge base.

Call it:

```
[
C_6:\neg ApplyDripMethod
]
```

Step 2: Resolve C1 with SoilDry

C1:

```
[
\neg SoilDry\lor IrrigationNeeded
]
```

C4:

```
[
SoilDry
]
```

Resolve them:

```
[
(\neg SoilDry\lor IrrigationNeeded),;SoilDry
]
```

The complementary literals are:

```
[
\neg SoilDry,;SoilDry
]
```

Therefore:

```
[
\boxed{IrrigationNeeded}
]
Call this:
[
C_7:IrrigationNeeded
]
```

Step 3: Resolve C2 with IrrigationNeeded

```
C2:
[
\neg IrrigationNeeded\lor\neg CropWheat\lor ApplyDripMethod
]
C7:
[
IrrigationNeeded
]
Resolve:
[
\neg IrrigationNeeded
]
with:
[
IrrigationNeeded
]
Result:
[
\neg CropWheat\lor ApplyDripMethod
]
Call this:
[
C_8:\neg CropWheat\lor ApplyDripMethod
]
```

Step 4: Resolve with CropWheat

```
C8:
[
\neg CropWheat\lor ApplyDripMethod
]
C5:
[
CropWheat
]
Resolve the complementary literals:
[
\neg CropWheat,;CropWheat
]
Result:
```



```
[
\boxed{ApplyDripMethod}
]
Call this:
[
C_9:ApplyDripMethod
]
```

Step 5: Resolve with the Negated Goal

We have:

```
[
C_9:ApplyDripMethod
]
```

and:

```
[
C_6:\neg ApplyDripMethod
]
```

Resolve:

```
[
ApplyDripMethod,;\neg ApplyDripMethod
]
```

Result:

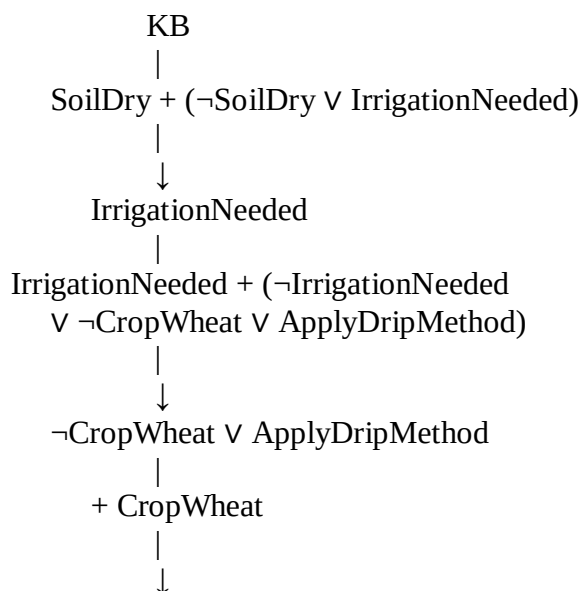
```
[
\boxed{\square}
]
```

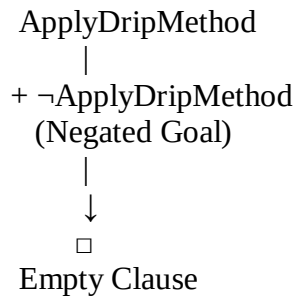
The empty clause ($\square$) represents a contradiction.

Therefore, the negated goal is impossible, and the original theorem is proved:

```
[
\boxed{KB\models ApplyDripMethod}
]
```

Resolution Proof Tree





Hence:

```
[
\boxed{\text{ApplyDripMethod is logically entailed.}}
]
```

(c) Removing the Fact SoilDry

Now suppose the fact:

```
[
SoilDry
]
```

is removed from the knowledge base.

The remaining facts are:

```
[
CropWheat
]
```

The rules remain:

```
[
\neg SoilDry \lor IrrigationNeeded
]
[
\neg IrrigationNeeded \lor \neg CropWheat \lor ApplyDripMethod
]
[
ApplyDripMethod \lor \neg CropWheat \lor CropAtRisk
]
```

Step 1: Can IrrigationNeeded Still Be Derived?

Previously, we had:

```
[
SoilDry
]
```

which allowed us to resolve:

```
[
\neg SoilDry \lor IrrigationNeeded
]
```

and derive:

```
[
IrrigationNeeded
]
```

But now SoilDry is no longer a fact.

Therefore, we cannot perform that resolution step.

So:

```
[  
\boxed{IrrigationNeeded\text{ cannot be derived}}  
]
```

Step 2: Can ApplyDripMethod Still Be Derived?

P2 requires:

```
[  
IrrigationNeeded\land CropWheat  
]
```

We still know:

```
[  
CropWheat=True  
]
```

but we do not know:

```
[  
IrrigationNeeded=True  
]
```

Therefore:

```
[  
\boxed{ApplyDripMethod\text{ cannot be derived}}  
]
```

Importantly, we also cannot conclude:

```
[  
\neg ApplyDripMethod  
]
```

Simply failing to prove something does not mean its negation is true.

Step 3: Does CropAtRisk Follow?

P3 is:

```
[  
\neg ApplyDripMethod\land CropWheat\rightarrow CropAtRisk  
]
```

In CNF:

```
[  
ApplyDripMethod\lor\neg CropWheat\lor CropAtRisk  
]
```

To derive CropAtRisk, we need:

```
[  
\neg ApplyDripMethod  
]
```

and:

```
[  
CropWheat  
]
```

We have:

```
[
CropWheat
]
but we do not have:
[
\neg ApplyDripMethod
]
Therefore:
[
\boxed{CropAtRisk\text{ does NOT follow}}
]
```

Important Logical Point

There is a difference between:

"ApplyDripMethod cannot be proved"

and

"ApplyDripMethod is false."

The removal of SoilDry only means that the KB no longer provides enough information to derive ApplyDripMethod. It does not automatically provide the negative fact:

```
[
\neg ApplyDripMethod
]
```

Therefore, CropAtRisk cannot be inferred.

Final Comparison

Knowledge Base IrrigationNeeded ApplyDripMethod CropAtRisk

Original KB	Derived	Derived	Not derived
SoilDry removed	Not derived	Not derived	Not derived

Final Conclusion

With SoilDry present:

```
[
SoilDry\rightarrow IrrigationNeeded
]
[
IrrigationNeeded\land CropWheat\rightarrow ApplyDripMethod
]
```

Therefore:

```
[
\boxed{ApplyDripMethod}
]
```

When SoilDry is removed, the reasoning chain breaks at IrrigationNeeded. Consequently, ApplyDripMethod cannot be derived.

However, **CropAtRisk** also does not follow, because the KB contains no fact or rule that derives:

```
[
\neg ApplyDripMethod
]
```

Thus, the correct conclusion is:

[
\boxed{\text{Without SoilDry, neither ApplyDripMethod nor CropAtRisk can be logically
concluded.}}
]

Case Study 4 — Wumpus World Knowledge Agent

The agent receives the following percepts:

- **Stench at [1,2]**
- **Breeze at [1,1]**
- **Glitter at [2,2]**

Rules:

5. $\text{Stench}[1,2] \rightarrow \text{WumpusAdjacent}[1,2]$
6. $\text{Breeze}[1,1] \rightarrow \text{PitAdjacent}[1,1]$
7. $\text{Glitter}[x,y] \rightarrow \text{Gold}[x,y]$
8. $\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

(a) Construct the Belief State and Apply Modus Ponens

Step 1: Initial Percepts

The agent initially knows:

$\text{Stench}[1,2]$ $\text{Breeze}[1,1]$ $\text{Glitter}[2,2]$

These form the initial **belief state**:

$\text{KB}_0 = \{\text{Stench}[1,2], \text{Breeze}[1,1], \text{Glitter}[2,2]\}$

Step 2: Apply Rule 1

Rule:

$\text{Stench}[1,2] \rightarrow \text{WumpusAdjacent}[1,2]$

We have:

$\text{Stench}[1,2]$

Therefore, by **Modus Ponens**:

WumpusAdjacent[1,2]

Step 3: Apply Rule 2

Rule:

$\text{Breeze}[1,1] \rightarrow \text{PitAdjacent}[1,1]$

We have:

$\text{Breeze}[1,1]$

Therefore:

$\text{PitAdjacent}[1,1]$

Step 4: Apply Rule 3

Rule:

$\text{Glitter}[x,y] \rightarrow \text{Gold}[x,y]$

Given:

$\text{Glitter}[2,2]$

Substitute:

$x=2, y=2$

Therefore:

$\text{Gold}[2,2]$

Step 5: Apply Rule 4

Rule:

$\neg \text{Wumpus}[x,y] \wedge \neg \text{Pit}[x,y] \rightarrow \text{Safe}[x,y]$

To apply this rule, we need explicit knowledge that a cell has:

$\neg \text{Wumpus}$

and

$\neg \text{Pit}$

No such negative facts are provided.

Therefore, **Rule 4 cannot currently be applied.**

Derived Knowledge

Percept/Rule	Derived Knowledge
Stench [1,2]	Wumpus is adjacent to [1,2]
Breeze [1,1]	Pit is adjacent to [1,1]
Glitter [2,2]	Gold is at [2,2]
No negative Wumpus/Pit facts	No Safe cell can be concluded

Therefore, the belief state contains:

Stench[1,2]Breeze[1,1]Glitter[2,2]WumpusAdjacent[1,2]PitAdjacent[1,1]Gold[2,2]

(b) Forward Chaining — Possible Wumpus and Pit Locations

We now use the standard Wumpus World assumption that a hazard indicated by a percept must be in **one of the adjacent cells**.

For a cell [x,y], the adjacent cells are:

[x-1,y],[x+1,y],[x,y-1],[x,y+1]

where valid grid cells are considered.

Step 1: Stench at [1,2]

We know:

Stench[1,2]

A stench means the Wumpus is in a neighboring cell.

The neighbors of [1,2] are:

- [1,1]
- [1,3]
- [2,2]

Therefore:

Wumpus $\in$ {[1,1],[1,3],[2,2]}

Step 2: Breeze at [1,1]

We know:

Breeze[1,1]

A breeze means a pit is in an adjacent cell.

The neighbors of [1,1] are:

- [1,2]
- [2,1]

Therefore:

$\text{Pit} \in \{[1,2], [2,1]\}$

Step 3: Glitter at [2,2]

We know:

Glitter[2,2]

Using Rule 3:

$\text{Glitter}[x,y] \rightarrow \text{Gold}[x,y]$

Therefore:

Gold[2,2]

Glitter gives information about the gold, **not about the Wumpus or pit.**

Forward-Chaining Summary

Stench[1,2]

↓

Wumpus adjacent to [1,2]

↓

Possible Wumpus:

[1,1], [1,3], [2,2]

Breeze[1,1]

↓

Pit adjacent to [1,1]

↓

Possible Pit:

[1,2], [2,1]

Glitter[2,2]
↓
Gold[2,2]

Final Possible Locations

Wumpus:

[1,1],[1,3],[2,2]

Pit:

[1,2],[2,1]

Gold:

[2,2]

Important: The given rules only say "adjacent." They do not provide enough information to identify one unique Wumpus or pit location.

(c) Is the Path [1,1] → [1,2] → [2,2] Safe?

The proposed path is:

[1,1]→[1,2]→[2,2]

We need to check every cell for possible hazards.

Step 1: Cell [1,1]

The agent starts at:

[1,1]

There is a **Breeze** at [1,1].

Therefore, a pit may be adjacent to [1,1]:

$\text{Pit} \in \{[1,2], [2,1]\}$

The current knowledge does **not** prove that [1,1] itself contains a pit.

However, Rule 4 requires explicit:

$\neg \text{Wumpus}[1,1] \wedge \neg \text{Pit}[1,1]$

These facts are not available.

So [1,1] is **not formally proven safe** by the given KB.

Step 2: Cell [1,2]

This is the most dangerous part of the proposed path.

There is a **Stench** at [1,2].

Also, because of the Breeze at [1,1]:

$Pit \in \{[1,2], [2,1]\}$

Therefore:

[1,2] may contain a Pit

Furthermore, the stench at [1,2] means the Wumpus is adjacent to [1,2]:

$Wumpus \in \{[1,1], [1,3], [2,2]\}$

Thus, the KB does not establish [1,2] as safe.

Step 3: Cell [2,2]

We know:

Gold[2,2]

But gold does **not** imply that the cell is safe.

In fact, from the stench at [1,2]:

$Wumpus \in \{[1,1], [1,3], [2,2]\}$

So [2,2] is a **possible Wumpus location**.

Therefore:

[2,2] is not proven safe

Final Safety Evaluation

Cell	Information	Safe?
[1,1]	Breeze indicates a possible adjacent pit	Not proven safe
[1,2]	Could contain a pit	Not safe to assume
[2,2]	Contains gold but could contain Wumpus	Not proven safe

Therefore, based strictly on the available knowledge:

The path $[1,1] \rightarrow [1,2] \rightarrow [2,2]$ cannot be confirmed safe.

The agent **should not take this path if safety requires certainty.**

Why?

The key logical point is:

Unknown $\square$ = Safe

Rule 4 says a cell is safe only when both are known:

$\neg \text{Wumpus}[x,y]$

and

$\neg \text{Pit}[x,y]$

But the current KB does not provide these negative facts for the cells on the proposed route.

Thus, although the agent knows that **Gold is at [2,2]**, it does not have enough information to safely reach it.

Final conclusion:

Gold location is known, but the proposed route is not logically guaranteed to be safe.